FULL STACK

LAB SHEET – 6

NAME – Vansh

CUID – CU24250033

SEC – B , ROLL – 64

Client/script.jss

```
const API_URL = 'http://localhost:5000/api/tasks';
const statusEl = document.getElementById('status');
const listEl = document.getElementById('taskList');

async function loadTasks() {
  statusEl.textContent = 'Loading...';
  try {
    const response = await fetch(API_URL);
    if (!response.ok) throw new Error(`Request failed: ${response.status}`);
    const tasks = await response.json();
    listEl.innerHTML = '';
    tasks.forEach(task => {
      const li = document.createElement('li');
      li.textContent = `${task.title} — ${task.completed ? 'Completed' : 'Pending'}`;
      listEl.appendChild(li);
    });
    statusEl.textContent = '';
  } catch (error) {
    statusEl.textContent = `Error: ${error.message}. Please check that the server is running.`;
  }
}
loadTasks();
```

client-react/src/App.jsx

```
import React, { useEffect, useState } from 'react';
import axios from 'axios';

const API_URL = import.meta.env.VITE_API_URL || 'http://localhost:5000/api';
const API_KEY = import.meta.env.VITE_API_KEY || 'api-lab-secret';

export default function App() {
  const [tasks, setTasks] = useState([]);
  const [title, setTitle] = useState('');
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');

  useEffect(() => { fetchTasks(); }, []);

  async function fetchTasks() {
    setLoading(true); setError('');
    try {
      const response = await fetch(`${API_URL}/tasks`);
      const data = await response.json();
      if (!response.ok) throw new Error(data.error || `Request failed: ${response.status}`);
      setTasks(data);
    } catch (err) { setError(err.message || 'Unable to load tasks'); }
    finally { setLoading(false); }
  }

  async function addTask(e) {
    e.preventDefault();
    if (!title.trim()) return;
```

```
  setError('');
  try {
    const response = await fetch(`${API_URL}/tasks`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json', 'x-api-key': API_KEY },
      body: JSON.stringify({ title: title.trim(), completed: false })
    });
    const data = await response.json();
    if (!response.ok) throw new Error(data.error || `Request failed: ${response.status}`);
    setTasks(prev => [...prev, data]);
    setTitle('');
  } catch (err) { setError(err.message || 'Unable to create task'); }
}

async function deleteTask(id) {
  setError('');
  try {
    const response = await fetch(`${API_URL}/tasks/${id}`, {
      method: 'DELETE', headers: { 'x-api-key': API_KEY }
    });
    const data = await response.json();
    if (!response.ok) throw new Error(data.error || `Request failed: ${response.status}`);
    setTasks(prev => prev.filter(task => task.id !== id));
  } catch (err) { setError(err.message || 'Unable to delete task'); }
}

async function addTaskWithAxios() {
  // Axios automatically parses JSON; errors are surfaced through catch (unlike fetch, which requires checking response.ok).
  try {
    const response = await axios.post(`${API_URL}/tasks`, { title: 'Axios example task', completed: false }, { headers: { 'x-api-key': API_KEY } });
```

```
      setTasks(prev => [...prev, response.data]);
    } catch (err) { setError(err.response?.data?.error || err.message); }
  }

  return <main className="container">
    <h1>Task Manager Mini App</h1>
    <p className="subtitle">React + Express REST API</p>
    <form onSubmit={addTask} className="form">
      <input value={title} onChange={e => setTitle(e.target.value)} placeholder="Enter a new task" />
      <button type="submit">Add Task</button>
    </form>
    <button className="secondary" onClick={addTaskWithAxios}>Add Axios Example</button>
    {loading && <p>Loading...</p>}
    {error && <p className="error">Error: {error}</p>}
    {!loading && !error && <ul className="tasks">
      {tasks.map(task => <li key={task.id}>
        <span>{task.title} <small>({task.completed ? 'Completed' : 'Pending'})</small></span>
        <button onClick={() => deleteTask(task.id)}>Delete</button>
      </li>)}
    </ul>}
  </main>;
}
```

Server/server.js

```
const express = require('express');
const cors = require('cors');
require('dotenv').config();

const app = express();
const PORT = process.env.PORT || 5000;
```

```
const API_KEY = process.env.API_KEY;

app.use(cors());
app.use(express.json());

// Global logger middleware: method, URL and timestamp.
app.use((req, res, next) => {
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.originalUrl}`);
  next();
});

const tasks = [
  { id: 1, title: 'Learn Express.js', completed: true },
  { id: 2, title: 'Build REST API', completed: true },
  { id: 3, title: 'Test API with Postman', completed: false },
  { id: 4, title: 'Create JavaScript client', completed: false },
  { id: 5, title: 'Create React client', completed: false }
];

function authCheck(req, res, next) {
  if (req.headers['x-api-key'] !== API_KEY) {
    return res.status(401).json({ error: 'Unauthorized: valid x-api-key is required' });
  }
  next();
}

function findTask(id) {
  return tasks.find(task => task.id === Number(id));
}

app.get('/api/tasks', (req, res) => res.json(tasks));
```

```
app.get('/api/tasks/:id', (req, res) => {
  const task = findTask(req.params.id);
  if (!task) return res.status(404).json({ error: 'Task not found' });
  res.json(task);
});

app.post('/api/tasks', authCheck, (req, res) => {
  const { title, completed = false } = req.body;
  if (!title || typeof title !== 'string') {
    return res.status(400).json({ error: 'title is required and must be a string' });
  }
  const task = { id: tasks.length ? Math.max(...tasks.map(t => t.id)) + 1 : 1, title, completed:
Boolean(completed) };
  tasks.push(task);
  res.status(201).json(task);
});

app.put('/api/tasks/:id', authCheck, (req, res) => {
  const task = findTask(req.params.id);
  if (!task) return res.status(404).json({ error: 'Task not found' });
  const { title, completed } = req.body;
  if (title !== undefined) task.title = title;
  if (completed !== undefined) task.completed = Boolean(completed);
  res.json(task);
});

app.delete('/api/tasks/:id', authCheck, (req, res) => {
  const index = tasks.findIndex(task => task.id === Number(req.params.id));
  if (index === -1) return res.status(404).json({ error: 'Task not found' });
  const deleted = tasks.splice(index, 1)[0];
```

```
  res.json({ message: 'Task deleted successfully', task: deleted });
});

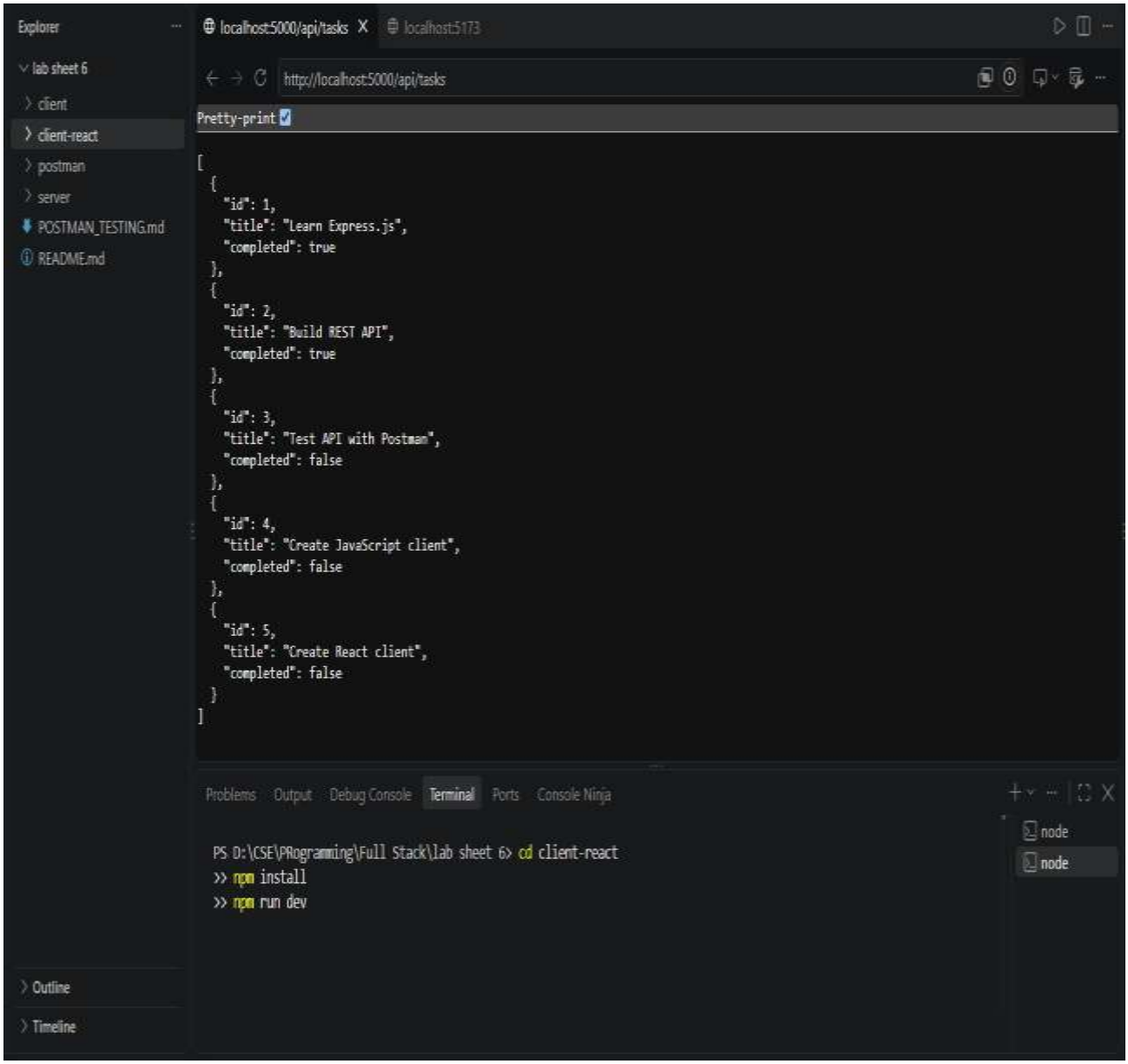
// 404 handler for unmatched URLs.
app.use((req, res) => res.status(404).json({ error: 'Route not found' }));

// Centralized error handler.
app.use((err, req, res, next) => {
  console.error(err.stack || err);
  res.status(err.status || 500).json({ error: err.message || 'Internal server error' });
});

app.listen(PORT, () => console.log(`API running at http://localhost:${PORT}`));
```

output




Explorer
lab sheet 6
client
client-react
postman
server
POSTMAN_TESTING.md
README.md
Outline
Timeline
localhost:5000/api/tasks
localhost:5173
http://localhost:5000/api/tasks
Pretty-print
[
{
"id": 1,
"title": "Learn Express.js",
"completed": true
},
{
"id": 2,
"title": "Build REST API",
"completed": true
},
{
"id": 3,
"title": "Test API with Postman",
"completed": false
},
{
"id": 4,
"title": "Create JavaScript client",
"completed": false
},
{
"id": 5,
"title": "Create React client",
"completed": false
}
]
Problems
Output
Debug Console
Terminal
Ports
Console Ninja
node
node
PS D:\CSE\PRogramming\Full Stack\lab sheet 6> cd client-react
>> npm install
>> npm run dev